MANAGEMENT DECISION · GENERAL EDITION

Security proposal for management

One shared foundation. Three complete protection packages.

Maximum 100 administrators | 2FA in every option | No additional servers to operate

Choose the common foundation plus one proposal. These are design recommendations, not implemented controls or a full cost quotation. Each item has two sentences and linked supporting evidence.

Strategy citations stay in this site and provide a return link. External sources open a new tab. The PDF links to the published general strategy; Markdown links to the separate sibling strategy export.

Contents

Common foundation

Proposal 1 · Lean AWS

Proposal 2 · AWS edge

Proposal 3 · Existing edge

01 · COMMON FOUNDATION

Protect the organization and every AWS account

01 Decision and scope

Approve the common foundation plus one of the three complete proposals below for a system with at most 100 administrators. [Strategy: Administrator access] Choose the lowest total cost that passes the security and emergency-load tests, without adding a server for the team to operate. [Strategy: Launch tests]

02 Separate production from experiments

Use separate AWS accounts for production, development, security tools, logs and backups so a development mistake cannot easily damage the live service. [Strategy: Account security] Keep the organization management account free of application workloads and restrict it to a very small administration team. [Strategy: Account security]

03 Rules nobody can casually switch off

Use AWS Organizations service control policies to block leaving the organization, disabling security checks and using unapproved regions. [AWS account safeguards] Test exceptions for essential AWS services and recovery access in development before applying these restrictions to production. [Strategy: Region safeguards]

04 Keep company data inside approved boundaries

Use resource control policies to limit outside access to supported storage, encryption and secret-management services. [AWS resource policies] List approved partners and AWS service exceptions explicitly so the protection does not interrupt legitimate work. [Strategy: Data boundaries]

05 Secure settings from the start

Block public storage, turn on disk encryption and permit only approved machine images when new resources are created. [Strategy: Secure defaults] Use organization policies and repeatable configuration to keep these settings consistent, with reviewed exceptions for the public website. [Strategy: Storage protection]

06 One identity per person, with two-factor sign-in

Give every administrator a named account, require two-factor authentication and reserve security keys or passkeys for the most powerful roles. [Strategy: Identity controls] Use short-lived access, review permissions quarterly and remove access promptly when someone leaves. [Strategy: Mandatory 2FA]

07 Protect the master keys

Protect the management account root user with hardware authentication and remove ordinary root credentials from member accounts where centralized root management is enabled. [AWS root guidance] Keep an independently authenticated emergency route and test that the organization policies do not block authorized recovery tasks. [Strategy: Root protection]

08 Keep evidence that an attacker cannot quietly erase

Send account activity and configuration history into a separate log account, with encryption and protected retention. [Strategy: Audit trail] Record who changed security settings or sensitive guidance while keeping passwords, tokens and unnecessary personal data out of logs. [Strategy: Logging]

09 Alerts instead of a staffed monitoring room

Use GuardDuty and Security Hub to collect security findings, with Inspector checking vulnerable software and Config checking account settings. [Strategy: Incident alerts] Page an assigned responder for likely compromise, send less urgent problems to a work queue and review routine findings weekly. [Strategy: Threat detection]

10 Backups that survive an account incident

Keep protected backup copies in a separate account and limit who can delete or change them. [Strategy: Recovery] Test restoration and approve the acceptable recovery time and data loss before launch rather than assuming a backup guarantees recovery. [AWS backup protection]

11 Approved changes, without stored cloud passwords

Use reviewed infrastructure code and deployment pipelines with temporary AWS credentials rather than permanent access keys. [Strategy: Deployment identity] Require a second person to approve high-risk account changes and alert when security controls are altered outside that process. [Strategy: Control-change alerts]

12 Make the full bill visible

Include security-service usage, log storage, identity licences, support and implementation time in each proposal budget instead of treating the access-service price as the total. [Strategy: Cost alerts] Set cost alerts and named owners, noting that a budget notification does not itself stop spending. [AWS WAF pricing]

13 Evidence before approval

Use the CIS controls and benchmarks, ISO 27001 and AWS security guidance as checklists for ownership, testing and evidence rather than as a claim of certification. [Strategy: Security frameworks] Management should approve an option only after a penetration test, recovery exercise and peak-traffic test resolve the launch-blocking risks. [Strategy: Launch tests]

Start with the lowest-complexity AWS package

01 **Best fit and trade-off**

Choose this as the budget-first candidate when direct access through the public AWS load balancer passes the peak-traffic and security tests. [Strategy: Network protection] It avoids an additional public edge vendor, but gives up the extra caching layer in Proposal 2 and the vendor support arrangement in Proposal 3. [Strategy: Emergency fallback]

02 **Filter attacks at the public entrance**

Put AWS WAF on the public load balancer with managed attack rules, allowed website paths and carefully tested request limits. [Strategy: Web attack protection] Use the included AWS network-attack protection as a baseline, without presenting it as a guarantee against every outage or application attack. [AWS DDoS protection]

03 **Keep emergency information reachable**

Do not block public guidance just because the visitor appears to be in another country, because legitimate roaming users can appear abroad. [Strategy: Web attack protection] Check submitted locations in the application and use stricter limits for suspicious writes rather than placing puzzles in front of emergency reading. [Strategy: Web attack protection]

04 **Private administrator access with two factors**

Use AWS Verified Access in front of the internal administrator load balancer and require two-factor sign-in through the chosen identity provider. [Strategy: Mandatory 2FA] Give each administrator only the permitted application actions and validate the signed identity rather than assuming a successful sign-in grants full authority. [Strategy: Application identity]

05 **Technical access without a jump server**

Budget for two managed Verified Access connections for the cluster and database, with local client software only for technical administrators. [Strategy: Administrator access] Use CloudShell inside the private network as a tested alternative where browser-based command-line work is sufficient, and verify region, identity and workstation compatibility before purchase. [AWS access client]

06 Separate public and administrative workloads

Keep application servers and databases on private networks and allow communication only between components that actually need it. [Strategy: Network protection] Use separate public and administrative workload groups, encrypted connections and default-deny network rules to limit the damage from a compromised component. [Strategy: Workload identities]

07 Small permissions for every component

Give each workload its own Kubernetes service identity and AWS permissions instead of sharing a powerful account. [Strategy: Workload identities] Give public services, administrative services and database migrations separate database logins with only the required reading or writing rights. [Strategy: Database permissions]

08 Protect sensitive information

Encrypt stored data and connections, keep passwords in Secrets Manager and rotate them through an approved process. [Strategy: Data protection] Collect only the personal information needed for the service and delete or aggregate it after the approved retention period. [Strategy: Data retention]

09 Deploy only checked software

Use maintained minimal container images, scan them for vulnerabilities and verify the signature of each released image. [Strategy: Container protection] Remove debugging access and privileged execution, enforce admission rules in the existing cluster and retain a tested rollback. [Strategy: Secure delivery]

10 Detect and recover inside the application

Send suspicious sign-ins, unexpected guidance changes and workload threats into the common alerting process, with a named responder and a short playbook. [Strategy: Incident alerts] Keep a separately hosted emergency information page and test switching to it when the normal application is unavailable. [Strategy: Emergency fallback]

11 Cost checkpoint, not a complete quote

At the cited example rates and 730 hours, one Verified Access web application plus two technical endpoints costs about USD 489 per month before usage and other services. [Verified Access pricing] A standard WAF example with one web ACL and ten billable rule entries adds USD 15 per month plus USD 0.60 per million inspected requests, with extra features and the administrator ACL priced separately. [AWS WAF pricing]

12 What the team still owns

This package adds no jump server, proxy server or separate monitoring server, but the existing application and Kubernetes workloads still need maintenance. [Strategy: Responsibilities] The team remains responsible for access policies, firewall tuning, client updates, incident response and a staged migration away from the existing edge arrangement. [Strategy: Access options]

13 Management approval condition

Approve this package only if the production-like traffic test shows that the public application and emergency fallback remain usable without an extra caching layer. [Strategy: Launch tests] If that test fails or the overall cost model favours caching, select Proposal 2 rather than removing essential security controls to stay within budget. [CloudFront pricing]

Add a managed edge for a surge-heavy public service

01 Best fit and trade-off

Choose this when reducing public traffic reaching the application is worth an additional managed delivery layer. [CloudFront pricing] It combines CloudFront, AWS WAF and Verified Access without a self-managed proxy, but its value depends on measured traffic and what can safely be cached. [Strategy: Access options]

02 Stop unwanted traffic before it reaches the application

Put CloudFront and AWS WAF in front of the public load balancer, using managed attack rules, approved routes and tested request limits. [Strategy: Web attack protection] Restrict the load balancer to CloudFront traffic and require a protected origin header so attackers cannot simply bypass the edge. [AWS origin protection]

03 Cache public content, never private responses

Cache public images, scripts and approved guidance content while keeping private responses, administration and personal location submissions out of shared caches. [CloudFront pricing] Define how quickly changed guidance becomes visible and rehearse emergency updates so caching cannot serve dangerously outdated instructions. [Strategy: Guidance integrity]

04 Protect visitors without blocking legitimate help

Keep public guidance available to roaming visitors and validate reported locations inside the application. [Strategy: Web attack protection] Apply stronger controls to suspicious submissions while using administrator location restrictions only as an extra check alongside mandatory two-factor authentication. [Strategy: Location controls]

05 Private administrator access with two factors

Use Verified Access for the internal administrator application, with mandatory two-factor sign-in and separate permissions for each administrator group. [Strategy: Mandatory 2FA] Verify the application identity and signed claims, keep sessions short and alert on unusual sign-ins or changes to published guidance. [Strategy: Application identity]

06 No additional server for technical access

Use managed Verified Access endpoints for the private cluster and database and install the client only on technical administrators' supported devices. [Strategy: Administrator access] Test the chosen identity provider and retain a separately authenticated CloudShell recovery path where region availability and the required commands have been confirmed. [AWS access client]

07 Contain an internal compromise

Keep application workloads and databases private, separating public services from administrative services with default-deny network rules. [Strategy: Network protection] Encrypt internal connections and allow each component to contact only its approved dependencies. [Strategy: Internal encryption]

08 Separate workload and database identities

Give each workload its own service account and narrow AWS permissions. [Strategy: Workload identities] Use separate database users for public services, administrators and migrations so a public-service weakness cannot inherit database-owner powers. [Strategy: Database permissions]

09 Reduce the information at risk

Encrypt data, keep credentials in Secrets Manager and limit access to personal information. [Strategy: Data protection] Apply approved retention and deletion rules, and verify that logs and CloudFront caches do not retain information they should not contain. [Strategy: Data retention]

10 Keep unsafe software out

Build from maintained minimal images, scan releases and accept only approved signed images in the existing cluster. [Strategy: Container protection] Run applications without unnecessary privileges, remove debugging ports and preserve a tested deployment rollback. [Strategy: Secure delivery]

11 Managed detection and a separate fallback

Feed edge, administrator and workload incidents into the common alert-only response process instead of building a staffed monitoring room. [Strategy: Incident alerts] Host the emergency page independently of the application cluster and rehearse how it stays reachable and current during an incident. [Strategy: Emergency fallback]

12 Compare total edge cost before buying

Budget about USD 489 per month for the example Verified Access configuration, then compare CloudFront pay-as-you-go with eligible flat-rate plans using normal and emergency traffic forecasts. [Verified Access pricing] Check which firewall features, logs and usage are included so the budget neither double-counts bundled protection nor treats the smallest plan as adequate for an emergency surge. [CloudFront pricing]

13 What the team still owns

AWS operates the delivery and access services, but the team still maintains the existing application, cluster policies, caching rules and administrator devices. [Strategy: Launch tests] Approve this package only after the traffic model, origin-bypass test, cache-update test and residency review justify its cost over Proposal 1. [AWS origin protection]

Retain Imperva where its contract justifies the cost

01 Best fit and trade-off

Choose this when retaining the existing Imperva arrangement avoids a costly migration or provides contractually valuable incident support. [Strategy: Imperva decision] Do not assume it is cheaper or more secure merely because it uses two vendors, and obtain a written price and service-scope confirmation. [Strategy: Web attack protection]

02 Use the existing edge as the first filter

Keep Imperva as the public entry point for web attacks, unwanted bots and traffic-flood protection under the selected service contract. [Strategy: Web attack protection] Confirm which rules, support services, traffic allowances and emergency response commitments are actually included rather than relying on product names or defaults. [Strategy: Imperva decision]

03 Do not let attackers go around Imperva

Restrict the public AWS load balancer to the approved Imperva paths and verify the origin connection with an approved protected mechanism. [Strategy: Origin protection] Keep AWS WAF at the origin for application-specific routes and attack rules, but do not blindly duplicate all filtering at both layers. [Strategy: Web attack protection]

04 Apply visitor checks where the real visitor is visible

Use Imperva for public client reputation and location checks because the origin otherwise sees the proxy address. [Strategy: Visitor reputation] Keep public emergency reading open to legitimate roaming users and validate submitted locations inside the application. [Strategy: Web attack protection]

05 Keep administration separate from the public edge

Use Verified Access with mandatory two-factor authentication in front of the internal administrator application rather than exposing the console through the public website. [Strategy: Mandatory 2FA] Give each administrator group only its own actions and validate signed identity claims in the application. [Strategy: Application identity]

06 Technical access without a server to maintain

Use two managed Verified Access endpoints for cluster and database access, with client software only for the technical administrators who need it. [Strategy: Administrator access] Validate identity-provider and workstation compatibility, and rehearse CloudShell recovery with independently protected emergency credentials. [AWS access client]

07 Contain attacks after the public boundary

Keep workloads and databases private and separate public services from administrative services with narrow network permissions. [Strategy: Network protection] Encrypt internal traffic and block unnecessary outbound communication so an infected component has fewer places to reach. [Strategy: Outbound restrictions]

08 Avoid shared powerful accounts

Use one Kubernetes identity and narrow AWS permissions per workload. [Strategy: Workload identities] Use different database logins for public services, administration and migrations, with the database-owner account kept out of normal application use. [Strategy: Database permissions]

09 Protect personal data across both providers

Encrypt data, store credentials securely and apply approved retention limits to the application and its logs. [Strategy: Data protection] Review where Imperva processes traffic and stores logs, and approve the required privacy terms and regional exceptions before deployment. [Strategy: Residency]

10 Enforce safe releases internally

Use maintained minimal images, vulnerability checks, signed releases and admission rules in the existing cluster. [Strategy: Container protection] Remove privileged execution and debugging access while retaining a tested rollback for application changes. [Strategy: Secure delivery]

11 One response process across two vendors

Send Imperva, AWS and application alerts to the same assigned responders with clear instructions for contacting each provider. [Strategy: Incident alerts] Test a separately hosted emergency page and a joint incident exercise so a provider hand-off does not delay recovery. [Strategy: Emergency fallback]

12 Price the contract and the duplicate inspection

Add the quoted Imperva service and support costs to the example USD 489 monthly Verified Access configuration and the selected AWS firewall, logging and detection charges. [Strategy: Imperva decision] Compare that total and the migration avoided against Proposals 1 and 2, including traffic overages, contract commitments and incident support. [Verified Access pricing]

13 What management must accept

No new self-managed connector or jump server is required, but the team still operates the existing workloads and coordinates policies and incidents across AWS and Imperva. [Strategy: Web attack protection] Approve this package only when the contract, tested protection and migration savings justify that additional cost and coordination. [Strategy: Responsibilities]